

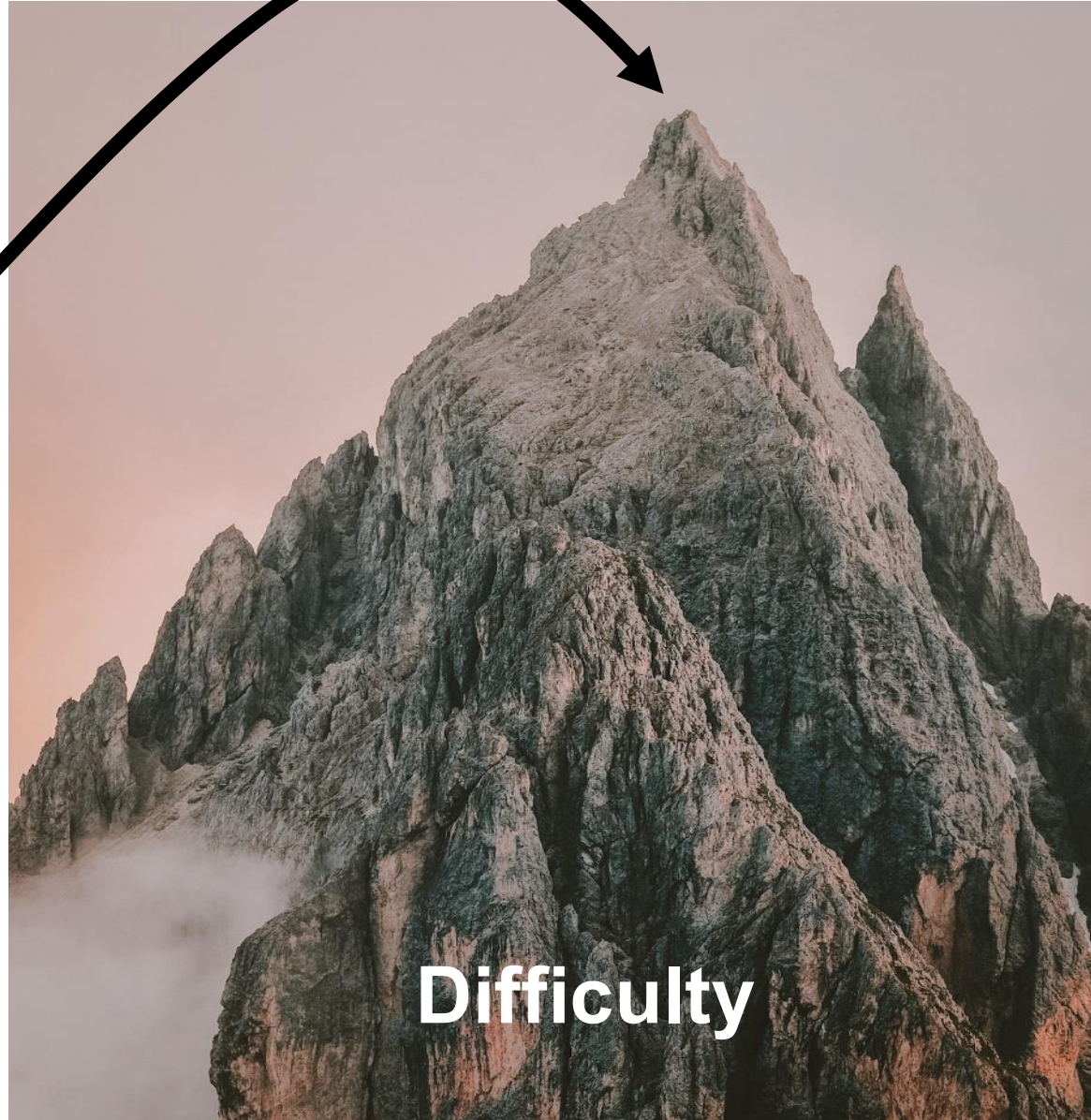
Week 10 – CuPy and GPU profiling

02613 Python and High-Performance Computing

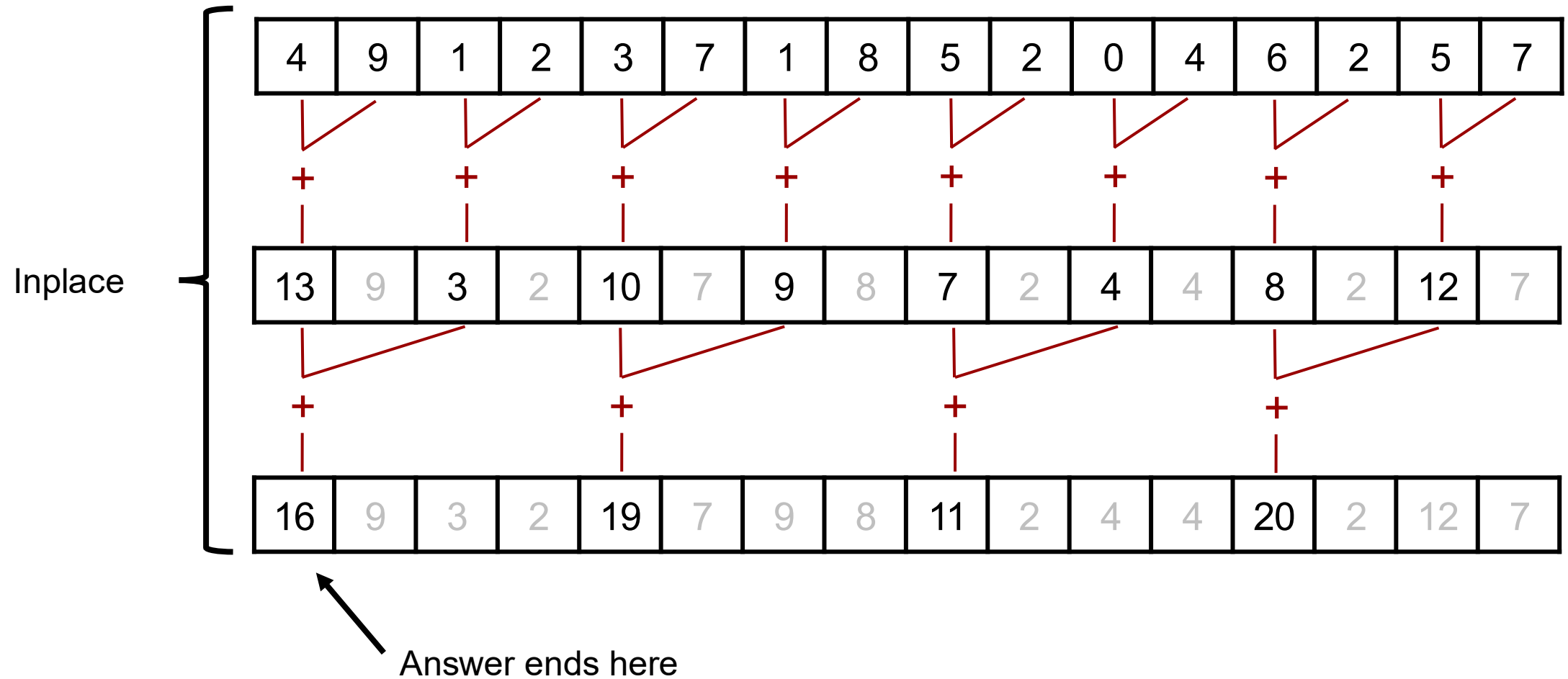
Today

- Revisit reductions – now on GPU!
- GPU profiling with NVIDIA nsys
- CuPy – CUDA NumPy

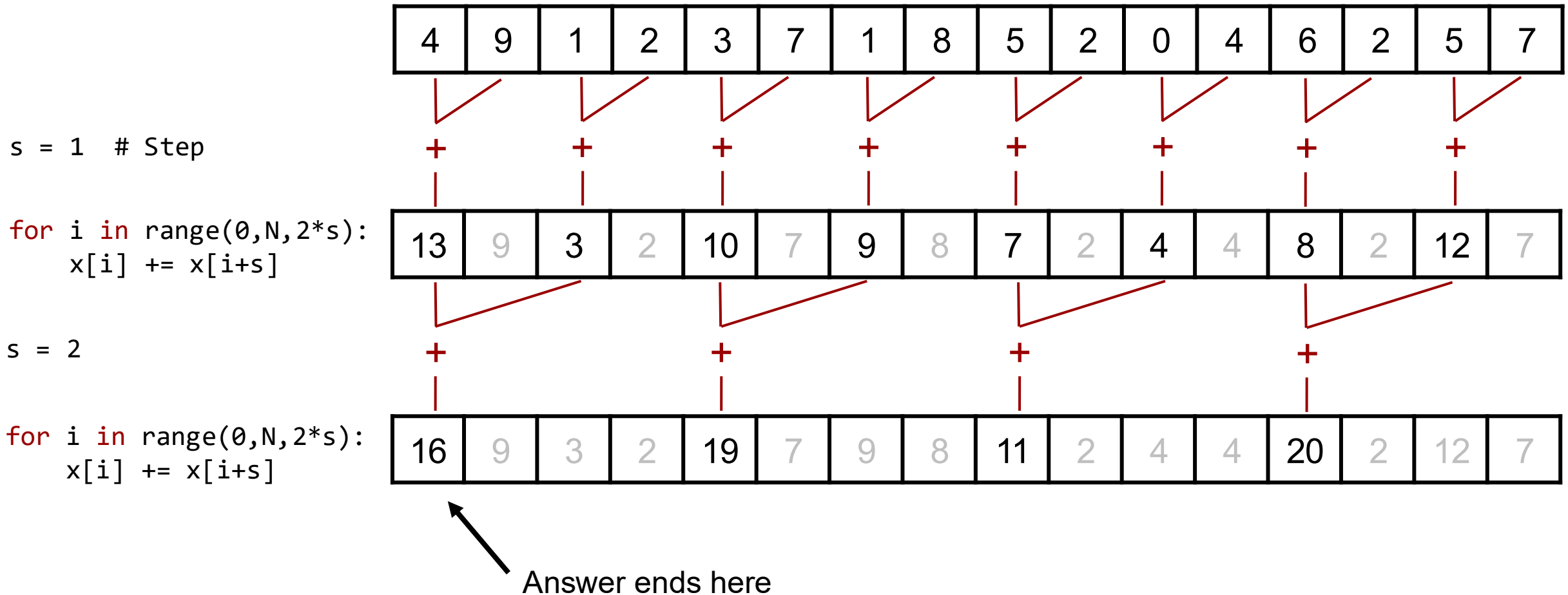
GPU Reductions



Reductions



Reductions





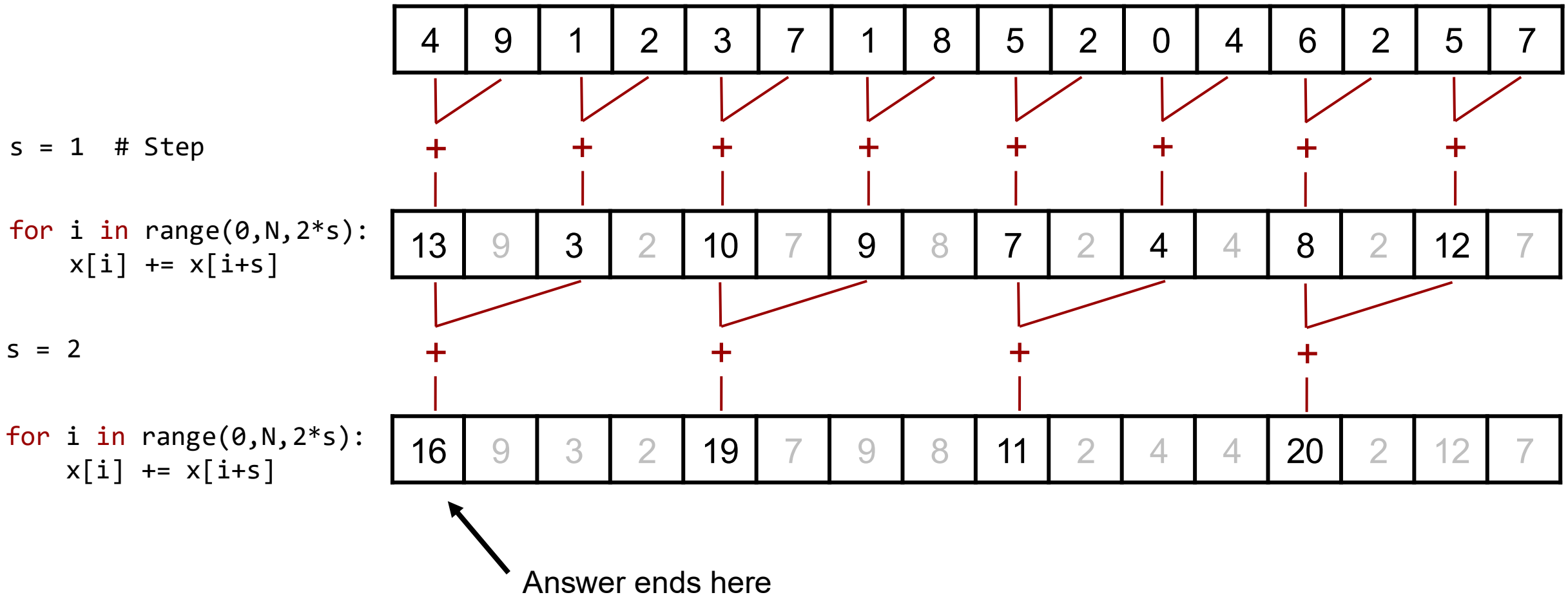
***somewhat outdated now,
but still relevant**

Optimizing Parallel Reduction in CUDA

Mark Harris
NVIDIA Developer Technology

From 2007, link in exercises

Reductions



Reductions

Problem: Synchronization

Before (on CPU):

Each level:

calls to `apply_async` in process pool
wait for calls to finish (synchronize)

Now (on GPU):

Each level:

call to kernel
CUDA kernels execute sequentially per default

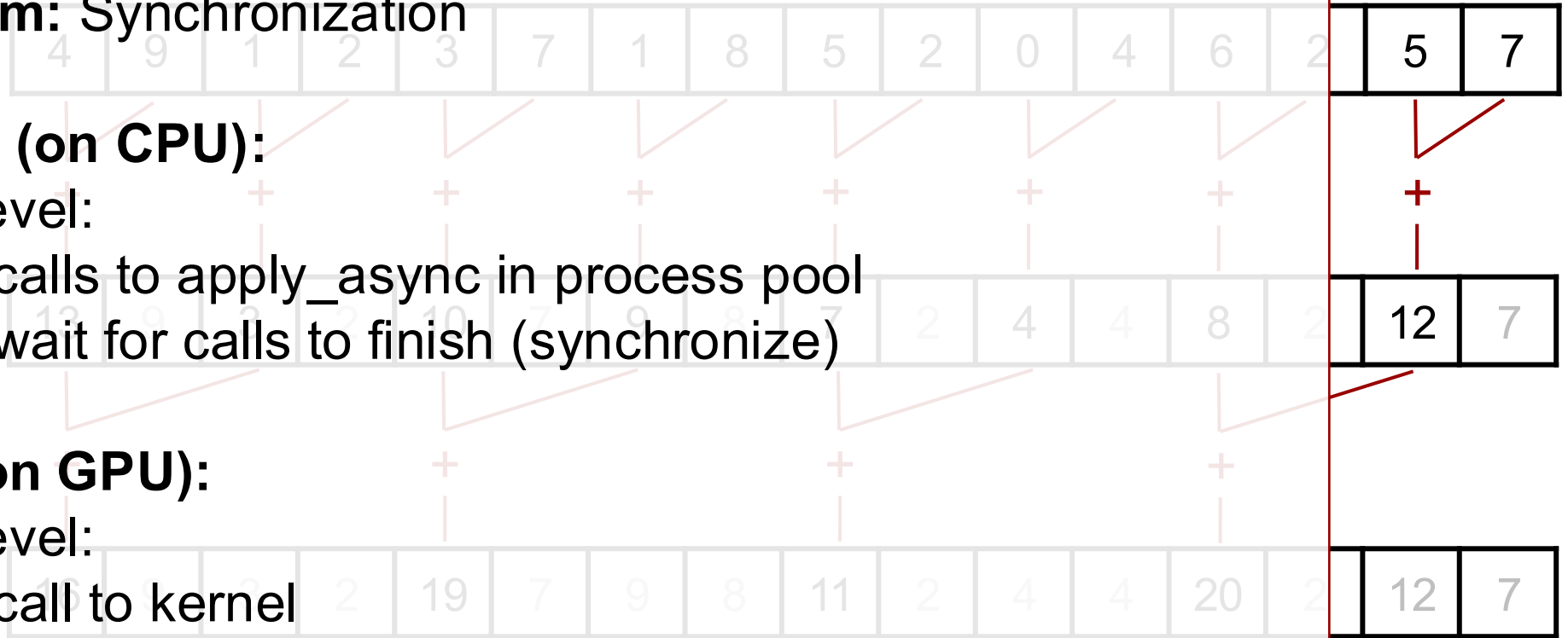
Answer ends here

```
s = 1 # Step
```

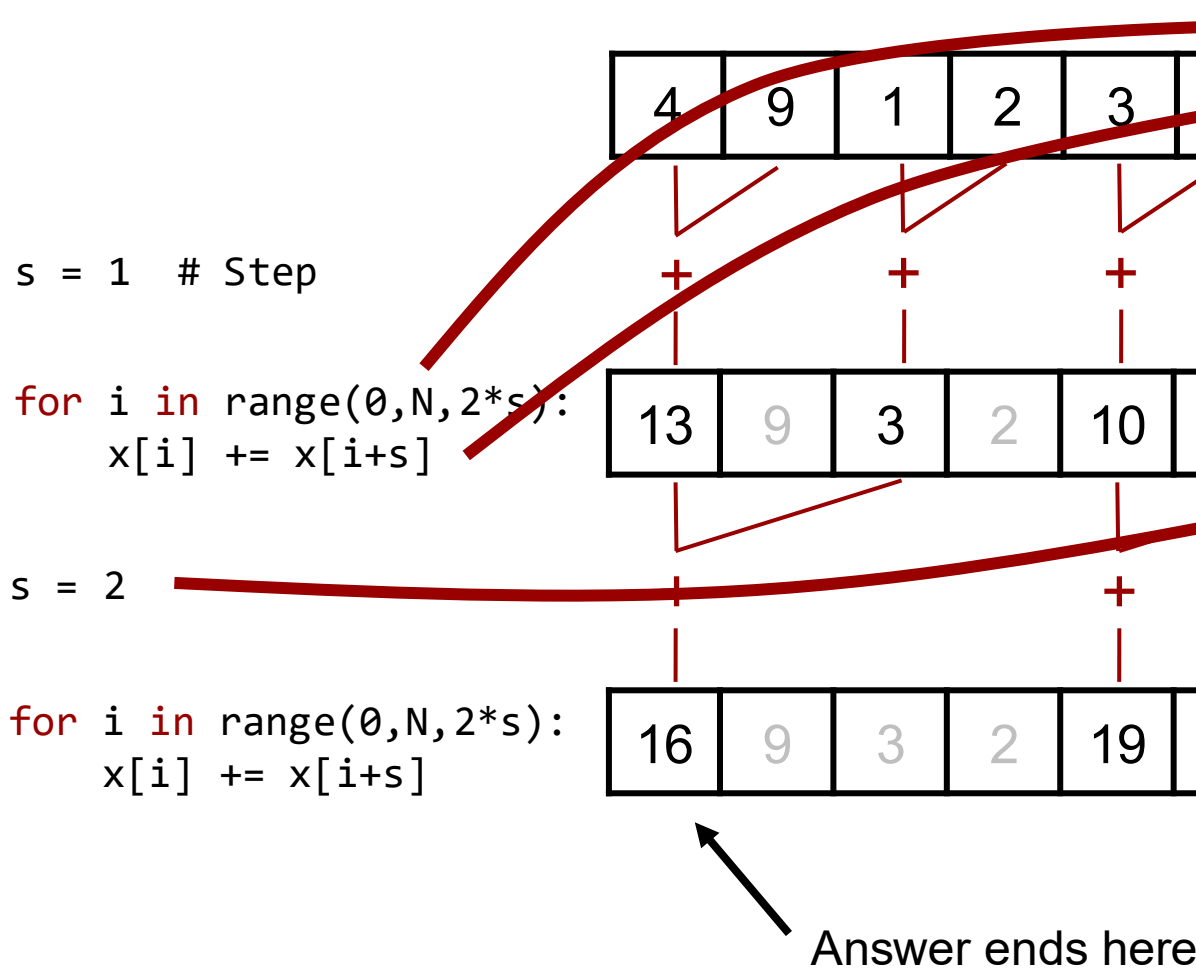
```
for i in range(0, N, 2*s):
    x[i] += x[i+s]
```

```
s = 2
```

```
for i in range(0, N, 2*s):
    x[i] += x[i+s]
```



Reductions



```
@cuda.jit
def reduce_kernel(x, s):
    i = cuda.grid(1)
    if i % (2 * s) == 0:
        x[i] = x[i] + x[i + s]

def reduce(x):
    n = len(x)
    tpb = 128 # Threads per block
    bpg = n // tpb # Blocks per grid
    s = 1
    while s < n:
        reduce_kernel[bpg, tpb](x, s)
        s = s * 2
```

Reductions

```
@cuda.jit
def reduce_kernel(x, s):
    i = cuda.grid(1)
    if i % (2 * s) == 0:
        x[i] = x[i] + x[i + s]
```

```
def reduce(x):
    n = len(x)
    # Threads per block
```

```
reduce_kernel[bpg, cpus]
s = s * 2
```

New problem: kernel launch overhead

s = 1 #

```
for i in range(0, N, s):
    x[i] += x[i+s]
```

s = 2

```
for i in range(0, N, 2*s):
    x[i] += x[i+s]
```

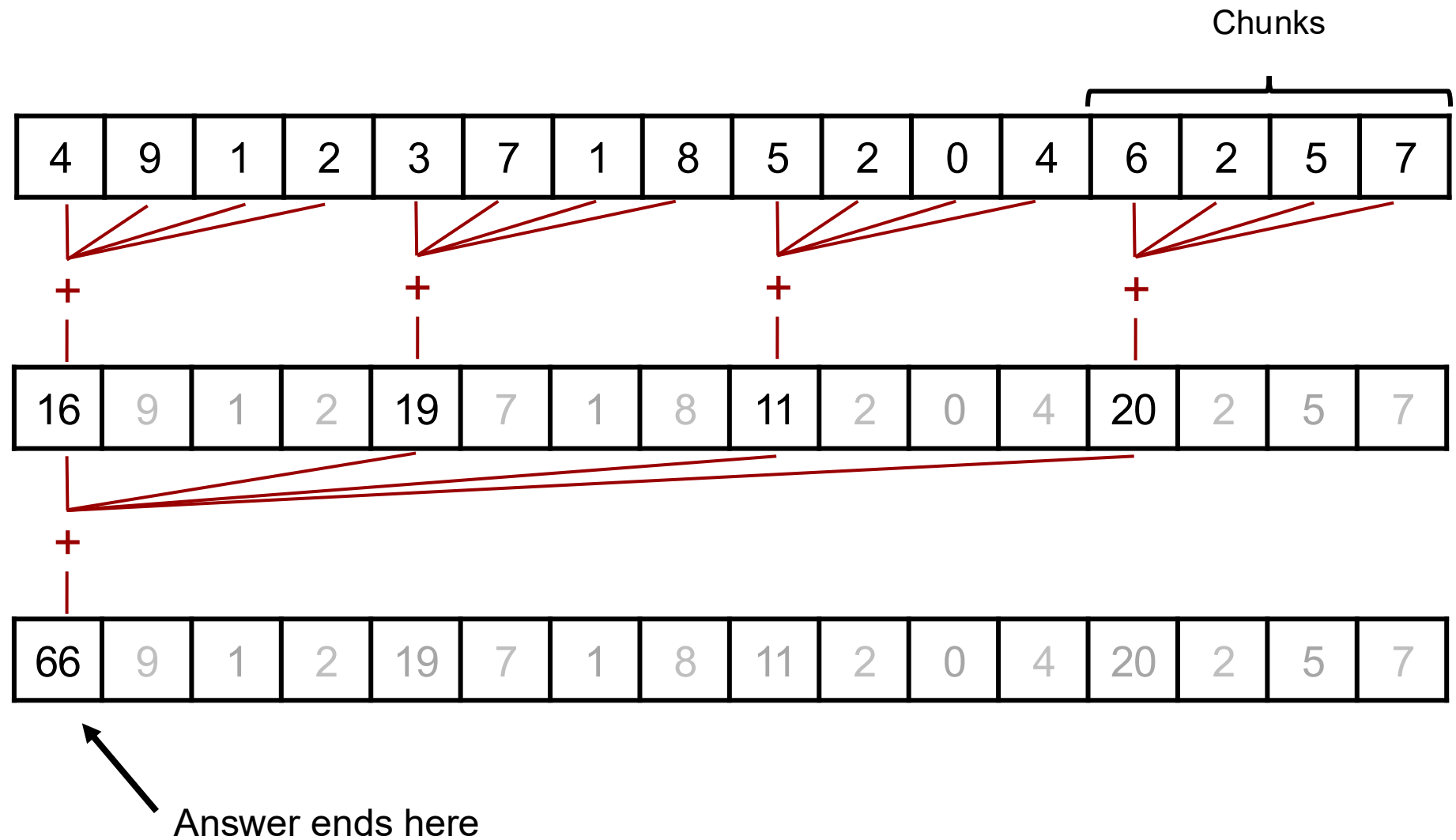
4	9	1	2	3
---	---	---	---	---

--	--	--	--	--

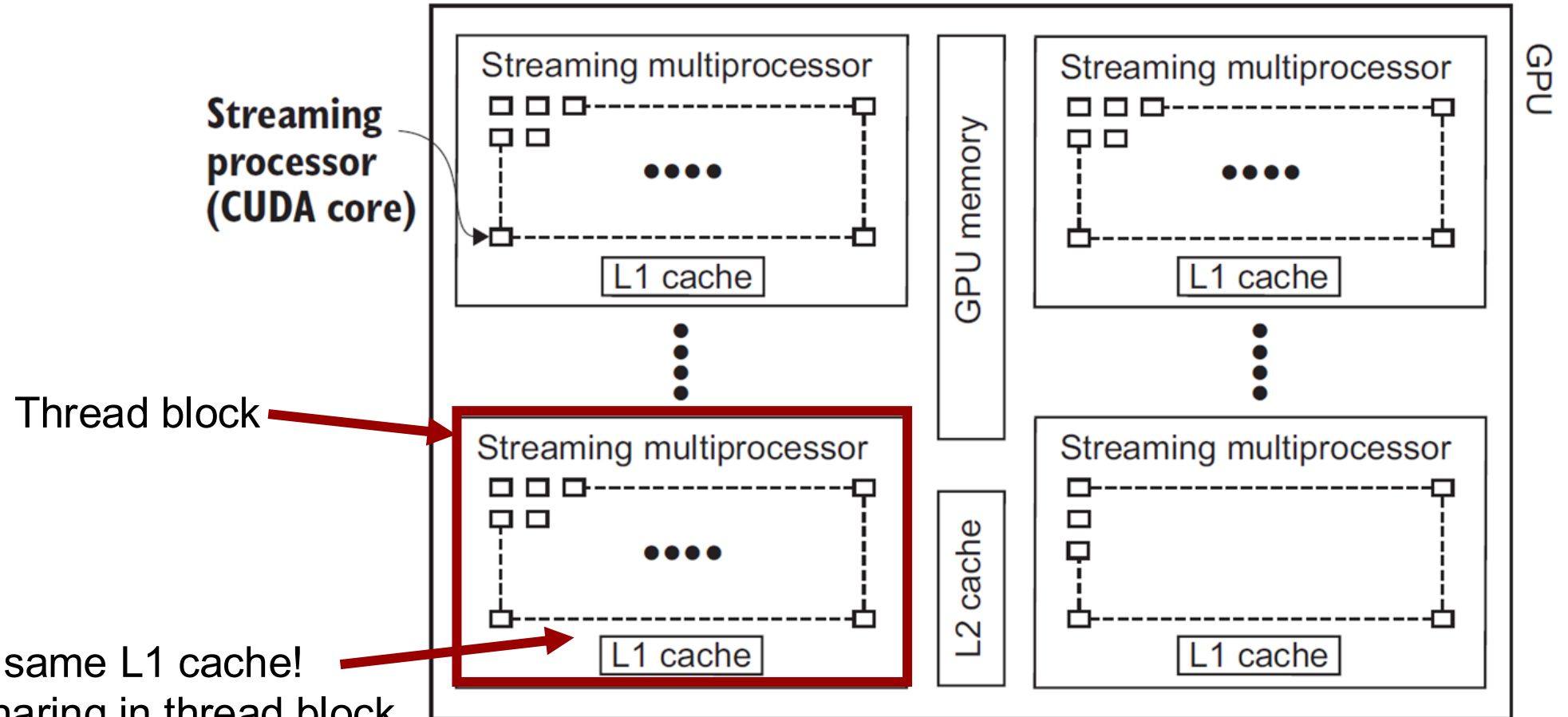
16	9	3	2	19
----	---	---	---	----

Answer ends here

On CPU: chunking to avoid overhead

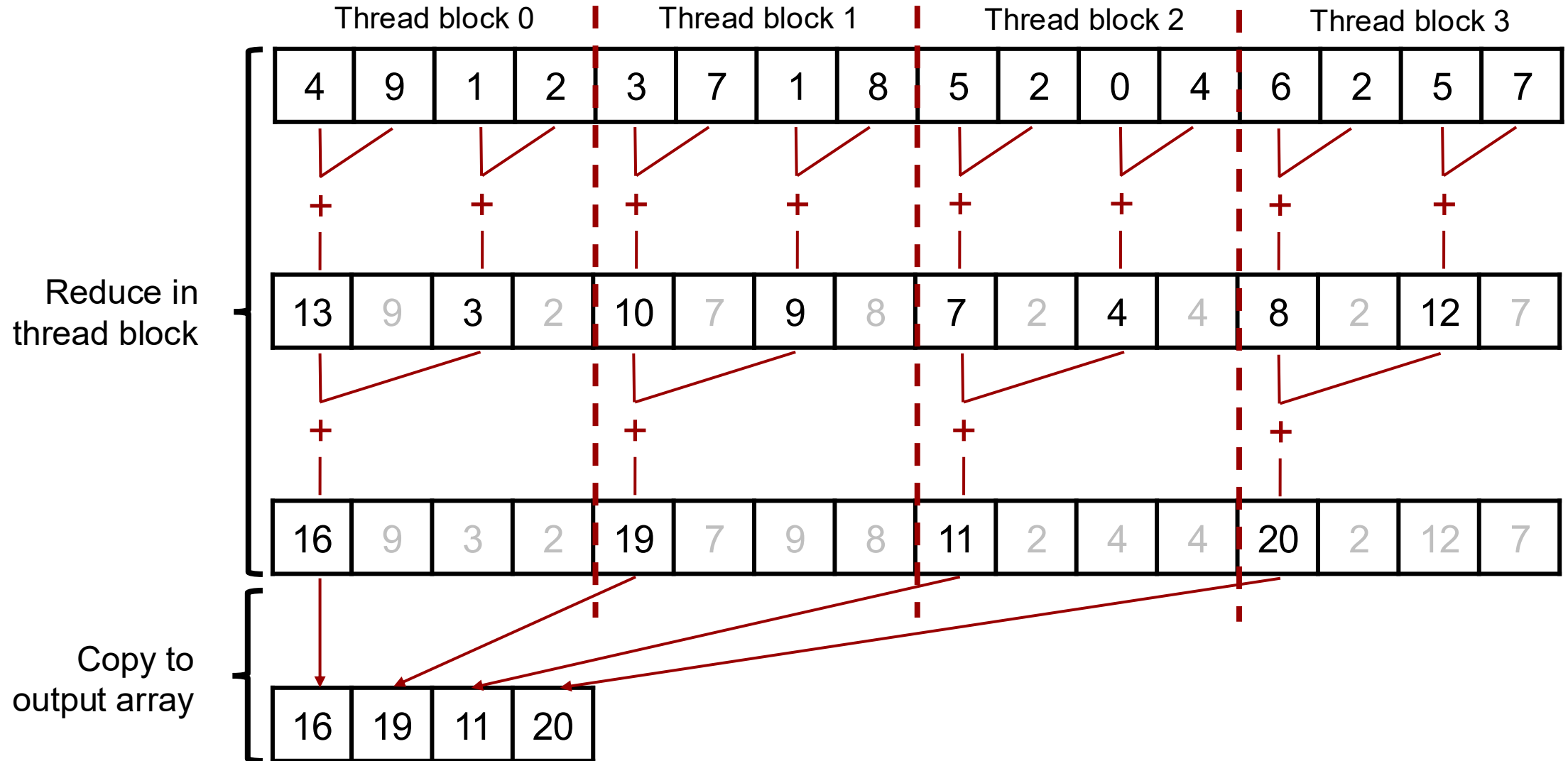


On GPU:



All cores share same L1 cache!
Efficient data sharing in thread block
Efficient synchronization in thread block

Block reduce kernel



```
@cuda.jit
def reduce_kernel(data, s):
    i = cuda.grid(1)
    if i % (2 * s) == 0:
        data[i] = data[i] + data[i + s]

def reduce(x):
    n = len(x)
    tpb = 128 # Threads per block
    bpg = n // tpb # Blocks per grid
    s = 1
    while s < n:
        reduce_kernel[bpg, tpb](x, s)
        s = s * 2
```

Single level per kernel

```
@cuda.jit
def block_reduce_kernel(data, out, n):
    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1) # Global index

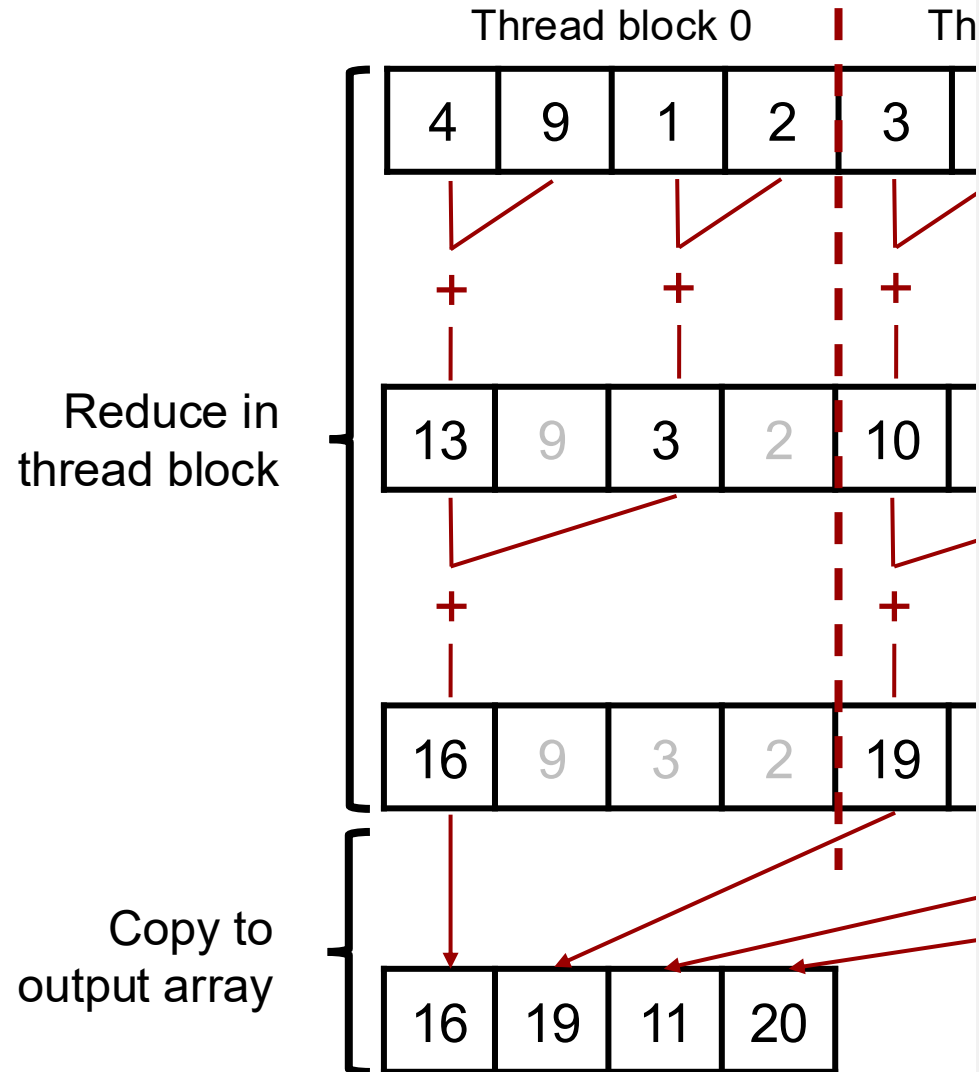
    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0 and i + s < n:
            data[i] = data[i] + data[i + s]
            s *= 2
        cuda.syncthreads() # Sync. block

    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = data[i]
```

Block per kernel

Warning: all threads in a block must reach `cuda.syncthreads()`, otherwise your kernel may become stuck on older GPUs!

Block reduce kernel



```
@cuda.jit
```

```
def block_reduce_kernel(data, out, n):
    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1)        # Global index
```

```
    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0 and i + s < n:
            data[i] = data[i] + data[i + s]
        s *= 2
        cuda.syncthreads() # Sync. block
```

```
    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = data[i]
```

```
def reduce(x):
```

```
    n = len(x)
    tpb = 128 # Threads per block
    bpg = get_grid(n, tpb) # Blocks per grid
    out = cuda.device_array(bpg, dtype=x.dtype)
    while bpg > 1:
```

```
        block_reduce_kernel[bpg, tpb](x, out, n)
        n = bpg
        bpg = get_grid(n, tpb)
        x[:n] = out[:n] # Copy back to input
```

```
    block_reduce_kernel[bpg, tpb](x, out, n)
    return out # Sum now in out[0]
```

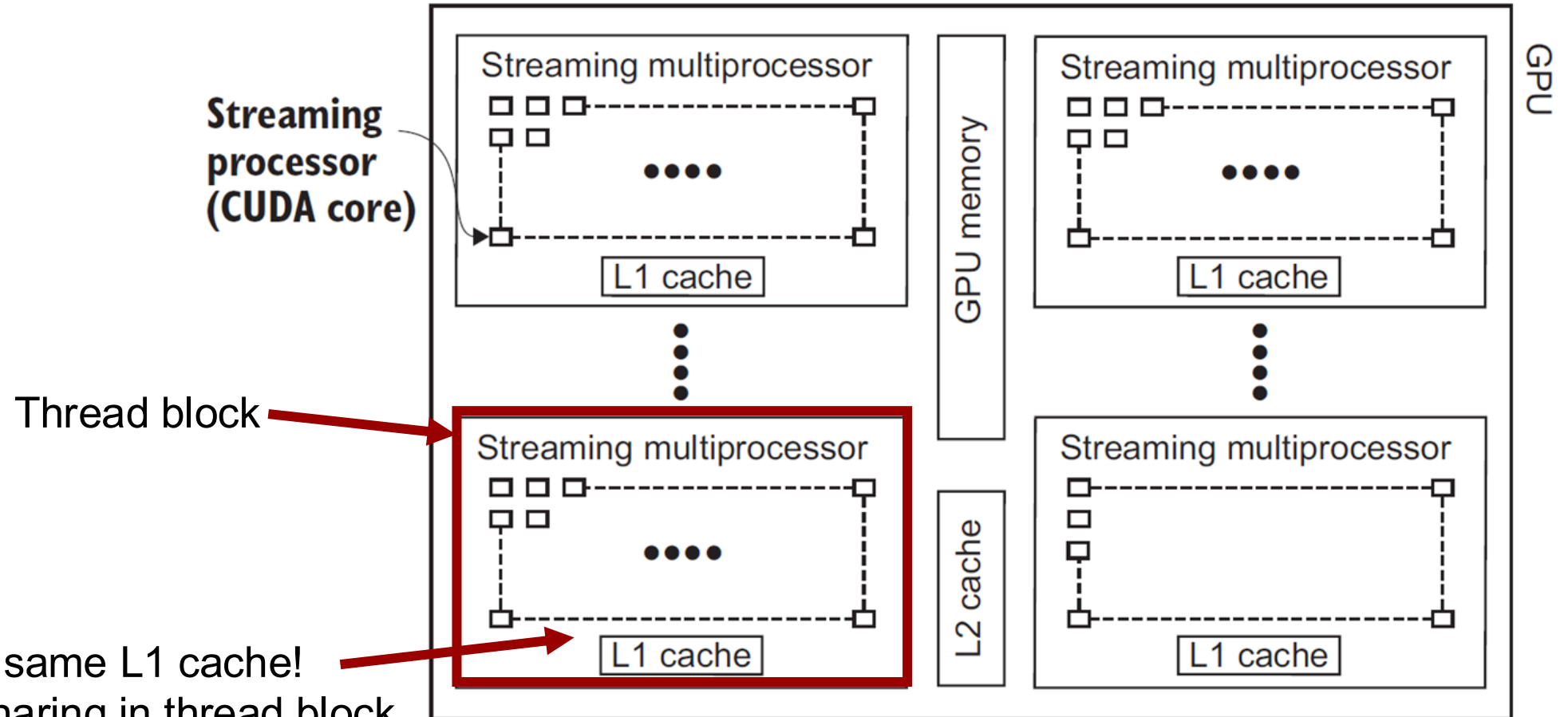
Kernel

Output array

While multiple blocks

Reduce final block

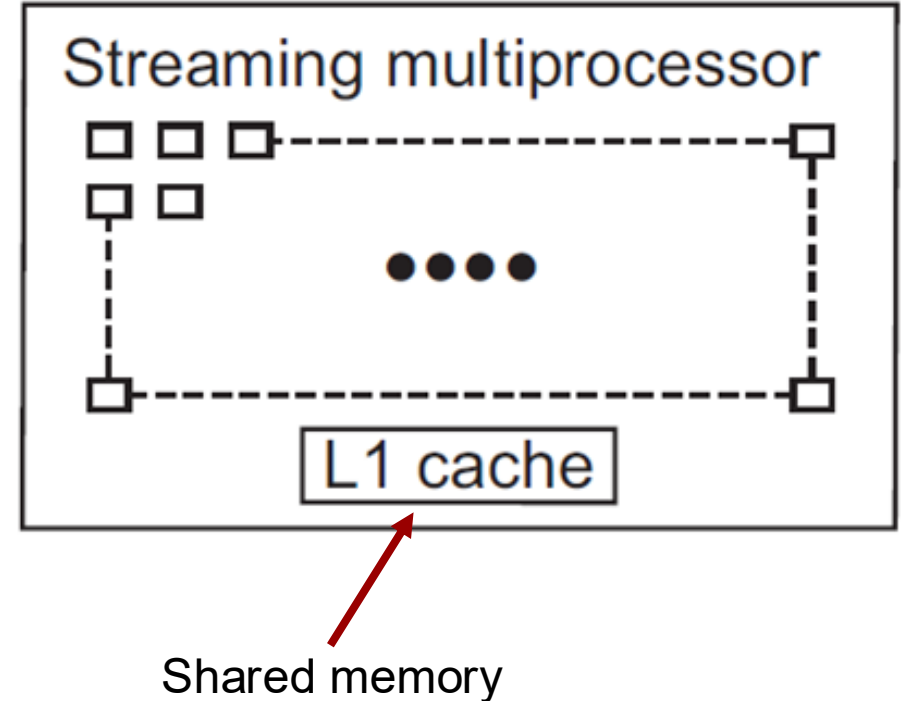
Efficient data sharing



- All cores share same L1 cache!
- (✓) Efficient data sharing in thread block
 - ✓ Efficient synchronization in thread block

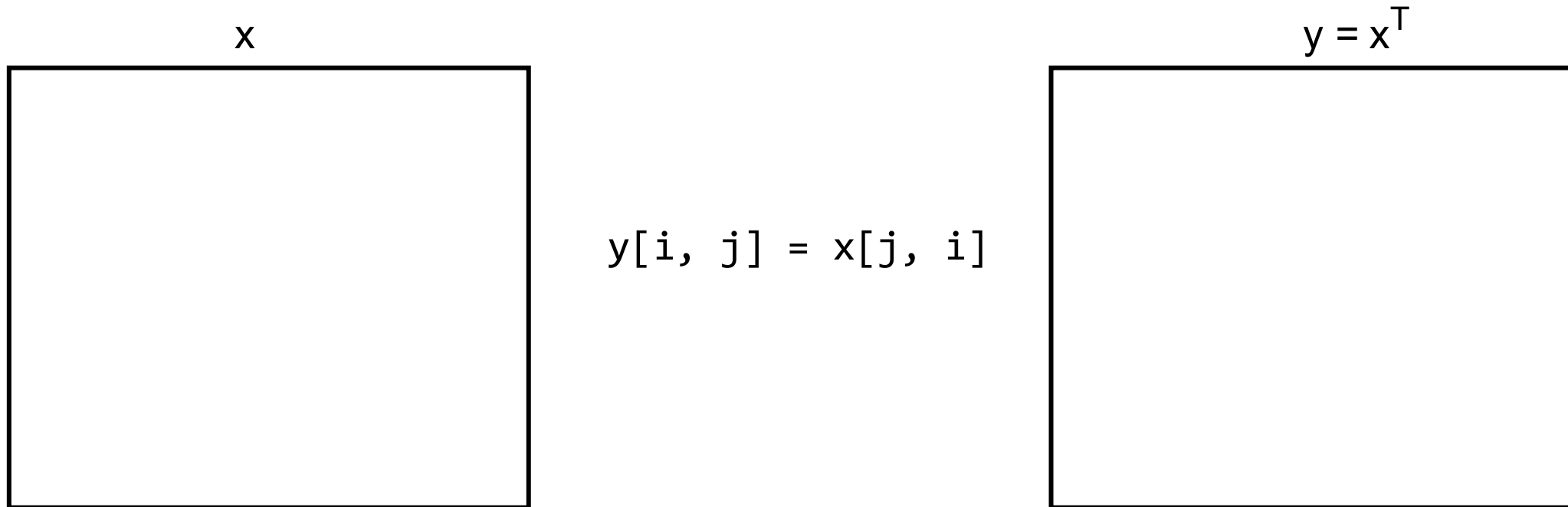
Shared memory

- Special memory local to each thread block
- Roughly same speed as L1 cache – so fast!
- Manually managed – we have full control
- Very limited (like L1 cache) < 100 KB
- Great when (and only when) threads access the same data
 - (a) repeatedly and/or
 - (b) in weird patterns!
 - I.e., we can't rely on normal L1 cache



Shared memory

Motivational example: matrix transpose



Problem: not matter what, either x or y will have bad data access w.r.t. cache.

Shared memory

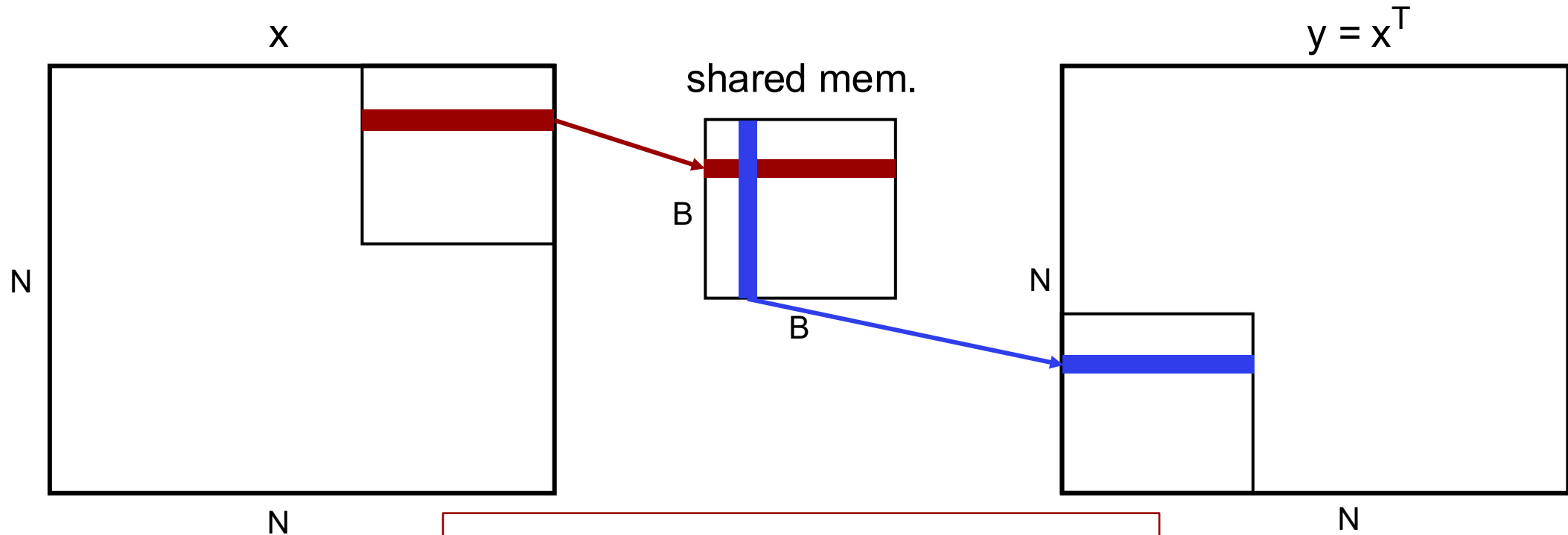
Motivational example: matrix transpose

Thread blocks with $B \times B$ threads

Threads copy x tile into shared mem row-wise

Threads synchronize (`cuda.syncthreads()`)

Threads copy shared mem columns into y row-wise



Solution: go through shared memory buffer. Cache friendly for both x and y .

Shared memory

Now:

Repeated (strided) access of same block of global memory

Repeated access



Instead:

1. Load block into shared memory
2. Do block reduction in shared memory
3. Write result back to global

```
@cuda.jit
def block_reduce_kernel(data, out, n):
    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1)        # Global index

    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0 and i + s < n:
            data[i] = data[i] + data[i + s]
            s *= 2
        cuda.syncthreads() # Sync. block

    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = data[i]
```

Shared memory

Declare shared array

Now:

Repeated (strided) access of same block of global memory

Instead:

1. Load block into shared memory
2. Do block reduction in shared memory
3. Write result back to global

```
@cuda.jit
def block_reduce_kernel(data, out, n):
    sdata = cuda.shared.array(TPB, dtype=out.dtype)

    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1)        # Global index

    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0 and i + s < n:
            data[i] = data[i] + data[i + s]
        s *= 2
        cuda.syncthreads() # Sync. block

    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = data[i]
```

Shared memory

Now:

Repeated (strided) access of same block of global memory

Load data from
global to shared



Instead:

1. Load block into shared memory
2. Do block reduction in shared memory
3. Write result back to global

```
@cuda.jit
def block_reduce_kernel(data, out, n):
    sdata = cuda.shared.array(TPB, dtype=out.dtype)

    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1)        # Global index

    # Each thread loads one element
    sdata[tid] = data[i] if i < n else 0.0
    cuda.syncthreads() # Ensure all are done

    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0 and i + s < n:
            data[i] = data[i] + data[i + s]
        s *= 2
        cuda.syncthreads() # Sync. block

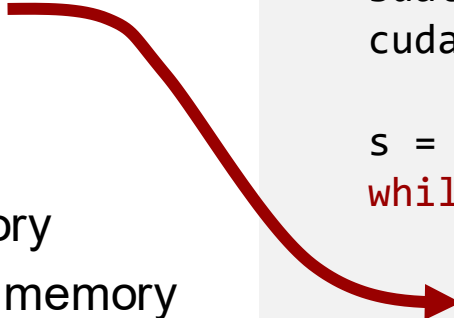
    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = data[i]
```

Shared memory

Now:

Repeated (strided) access of same block of global memory

Reduce in shared data



Instead:

1. Load block into shared memory
2. Do block reduction in shared memory
3. Write result back to global

```
@cuda.jit
def block_reduce_kernel(data, out, n):
    sdata = cuda.shared.array(TPB, dtype=out.dtype)

    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1)        # Global index

    # Each thread loads one element
    sdata[tid] = data[i] if i < n else 0.0
    cuda.syncthreads() # Ensure all are done

    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0:
            sdata[tid] = sdata[tid] + sdata[tid + s]
        s *= 2
        cuda.syncthreads() # Sync. block

    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = data[i]
```



Shared memory

Now:

Repeated (strided) access of same block of global memory

Instead:

1. Load block into shared memory
2. Do block reduction in shared memory
3. Write result back to global

```
@cuda.jit
def block_reduce_kernel(data, out, n):
    sdata = cuda.shared.array(TPB, dtype=out.dtype)

    tid = cuda.threadIdx.x # Index in block
    i = cuda.grid(1)        # Global index

    # Each thread loads one element
    sdata[tid] = data[i] if i < n else 0.0
    cuda.syncthreads() # Ensure all are done

    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0:
            sdata[tid] = sdata[tid] + sdata[tid + s]
        s *= 2
        cuda.syncthreads() # Sync. block

    if tid == 0: # First thread in block
        out[cuda.blockIdx.x] = sdata[0]
```



Write from shared data to out

Quiz time!

GPU profiling with NVIDIA nsys

Recall



Profiling: function level

```
import time

def fast(x):
    result = x + 2
    return result

def slow(x):
    # Job security
    time.sleep(3)
    result = x * x
    return result

print(fast(10))
print(slow(10))
```

script.py

```
$ python -m cProfile -s cumulative script.py
12
100
8 function calls in 3.000 seconds
```

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	3.000	3.000	{built-in method builtins.exec}
1	0.000	0.000	3.000	3.000	script.py:1(<module>)
1	0.000	0.000	3.000	3.000	script.py:8(slow)
1	3.000	3.000	3.000	3.000	{built-in method time.sleep}
2	0.000	0.000	0.000	0.000	{built-in method builtins.print}
1	0.000	0.000	0.000	0.000	{method 'disable' of...
1	0.000	0.000	0.000	0.000	script.py:3(fast)

CUDA Profiling with nsys

```
import numpy as np
from numba import cuda
```

```
@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]
```

```
# Make random test data
n = 1_000_000
x = np.random.rand(n)
y = np.empty_like(x)
```

```
# Define compute grid and run
tpb = 1024
bpg = (n + (tpb - 1)) // tpb
double_kernel[bpg, tpb](x, y)
```

double.py

```
$ nsys profile -o double_prof python double.py
```

Profile
command

Output file

What to profile

CUDA Profiling with nsys

```
import numpy as np
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]

# Make random test data
n = 1_000_000
x = np.random.rand(n)
y = np.empty_like(x)

# Define compute grid and run
tpb = 1024
bpg = (n + (tpb - 1)) // tpb
double_kernel[bpg, tpb](x, y)
```

double.py

```
$ nsys profile -o double_prof python double.py
WARNING: CPU IP/backtrace sampling not supported, disabling.
Try the 'nsys status --environment' command to learn more.

...

Generating '/tmp/nsys-report-b66f.qdstrm'
[1/1] [=====100%] double_prof.nsys-rep
Generated:
    /zhome/9d/d/98006/Documents/CUDA/double_prof.nsys-rep
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** OS Runtime Summary (osrtsum):
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
-----	-----	-----	-----	-----	-----	-----	-----	-----
71.9	442603667	19	23294929.8	1542361.0	4262	142230675	42617282.8	poll
12.7	77885716	657	118547.5	113114.0	1521	811733	61509.5	open64
12.1	74219824	781	95031.8	11792.0	1008	30527903	1118696.4	ioctl
1.5	9075649	824	11014.1	3821.0	1107	294676	22469.8	read
0.5	3376140	39	86567.7	4908.0	1708	933793	214053.1	fopen
0.3	1829193	127	14403.1	3452.0	1001	1163003	102959.9	mmap64
0.3	1674797	4	418699.3	337530.5	1272	998464	499853.3	fcntl

```
...
```

Can basically ignore this

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** CUDA API Summary (cudaapisum):
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
-----	-----	-----	-----	-----	-----	-----	-----	-----
40.3	5334575	2	2667287.5	2667287.5	1575577	3758998	1543911.8	cuMemcpyDtoH_v2
29.3	3878812	2	1939406.0	1939406.0	1875200	2003612	90801.0	cuMemcpyHtoD_v2
25.6	3385360	1	3385360.0	3385360.0	3385360	3385360	0.0	cuModuleLoad...
2.1	274899	2	137449.5	137449.5	123429	151470	19828.0	cuMemAlloc_v2
1.1	140527	1	140527.0	140527.0	140527	140527	0.0	cuLinkComplete
0.7	92664	384	241.3	214.0	117	2240	136.4	cuGetProcAdd...
0.5	60720	1	60720.0	60720.0	60720	60720	0.0	cuLinkCreate_v2
0.3	42001	1	42001.0	42001.0	42001	42001	0.0	cuMemGetInfo_v2
0.2	24852	1	24852.0	24852.0	24852	24852	0.0	cuLaunchKernel
0.0	1580	1	1580.0	1580.0	1580	1580	0.0	cuInit
0.0	1159	1	1159.0	1159.0	1159	1159	0.0	cuLinkDestroy
0.0	348	1	348.0	348.0	348	348	0.0	cuDeviceGetU...
0.0	226	1	226.0	226.0	226	226	0.0	cuModuleGetL...

```
...
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

Total time % for this section (sum):

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
40.3	5334575	2	2667287.5	2667287.5	1575577	3758998	1543911.8	cuMemcpyDtoH_v2
29.3	3878812	2	1939406.0	1939406.0	1875200	2003612	90801.0	cuMemcpyHtoD_v2
25.6	3385360	1	3385360.0	3385360.0	3385360	3385360	0.0	cuModuleLoad...
2.1	274899	2	137449.5	137449.5	123429	151470	19828.0	cuMemAlloc_v2
1.1	140527	1	140527.0	140527.0	140527	140527	0.0	cuLinkComplete
0.7	92664	384	241.3	214.0	117	2240	136.4	cuGetProcAdd...
0.5	60720	1	60720.0	60720.0	60720	60720	0.0	cuLinkCreate_v2
0.3	42001	1	42001.0	42001.0	42001	42001	0.0	cuMemGetInfo_v2
0.2	24852	1	24852.0	24852.0	24852	24852	0.0	cuLaunchKernel
0.0	1580	1	1580.0	1580.0	1580	1580	0.0	cuInit
0.0	1159	1	1159.0	1159.0	1159	1159	0.0	cuLinkDestroy
0.0	348	1	348.0	348.0	348	348	0.0	cuDeviceGetU...
0.0	226	1	226.0	226.0	226	226	0.0	cuModuleGetL...
...								

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** CUDA API Summary (cudaapisum):
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
-----	-----	-----	-----	-----	-----	-----	-----	-----
40.3	5334575	2	2667287.5	2667287.5	1575577	3758998	1543911.8	cuMemcpyDtoH_v2
29.3	3878812	2	1939406.0	1939406.0	1875200	2003612	90801.0	cuMemcpyHtoD_v2
25.6	3385360	1	3385360.0	3385360.0	3385360	3385360	0.0	cuModuleLoad...
2.1	274899	2	137449.5	137449.5	123429	151470	19828.0	cuMemAlloc_v2
1.1	140527	1	140527.0	140527.0	140527	140527	0.0	cuLinkComplete
0.7	92664	384	241.3	214.0	117	2240	136.4	cuGetProcAdd...
0.5	60720	1	60720.0	60720.0	60720	60720	0.0	cuLinkCreate_v2
0.3	42001	1	42001.0	42001.0	42001	42001	0.0	cuMemGetInfo_v2
0.2	24852	1	24852.0	24852.0	24852	24852	0.0	cuLaunchKernel
0.0	1580	1	1580.0	1580.0	1580	1580	0.0	cuInit
0.0	1159	1	1159.0	1159.0	1159	1159	0.0	cuLinkDestroy
0.0	348	1	348.0	348.0	348	348	0.0	cuDeviceGetU...
0.0	226	1	226.0	226.0	226	226	0.0	cuModuleGetL...

```
...
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** CUDA API Summary (cudaapisum):
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)
40.3	5334575	2	2667287.5	2667287
29.3	3878812	2	1939406.0	1939406
25.6	3385360	1	3385360.0	3385360
2.1	274899	2	137449.5	137449
1.1	140527	1	140527.0	140527
0.7	92664	384	241.3	214
0.5	60720	1	60720.0	60720
0.3	42001	1	42001.0	42001
0.2	24852	1	24852.0	24852
0.0	1580	1	1580.0	1580
0.0	1159	1	1159.0	1159
0.0	348	1	348.0	348
0.0	226	1	226.0	226

```
...
```

```
import numpy as np
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]
```

```
# Make random test data
```

```
n = 1_000_000
```

```
x = np.random.rand(n)
```

```
y = np.empty_like(x)
```

```
# Define compute grid and run
```

```
tpb = 1024
```

```
bpg = (n + (tpb - 1)) // tpb
```

```
double_kernel[bpg, tpb](x, y)
```

```
Name
```

```
-----
cuMemcpyDtoH_v2
cuMemcpyHtoD_v2
cuModuleLoad...
cuMemAlloc_v2
cuLinkComplete
cuGetProcAdd...
cuLinkCreate_v2
cuMemGetInfo_v2
cuLaunchKernel
cuInit
cuLinkDestroy
cuDeviceGetU...
cuModuleGetL...
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** CUDA API Summary (cudaapisum):
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
-----	-----	-----						-----
40.3	5334575	2	2667287.5	2667287.5	1575577	3758998	1543911.8	cuMemcpyDtoH_v2
29.3	3878812	2	1939406.0	1939406.0	1875200	2003612	90801.0	cuMemcpyHtoD_v2
25.6	3385360	1	3385360.0	3385360.0	3385360	3385360	0.0	cuModuleLoad...
2.1	274899	2	137449.5	137449.5	123429	151470	19828.0	cuMemAlloc_v2
1.1	140527	1	140527.0	140527.0	140527	140527	0.0	cuLinkComplete
0.7	92664	384	241.3	214.0	117	2240	136.4	cuGetProcAdd...
0.5	60720	1	60720.0	60720.0	60720	60720	0.0	cuLinkCreate_v2
0.3	42001	1	42001.0	42001.0	42001	42001	0.0	cuMemGetInfo_v2
0.2	24852	1	24852.0	24852.0	24852	24852	0.0	cuLaunchKernel
0.0	1580	1	1580.0	1580.0	1580	1580	0.0	cuInit
0.0	1159	1	1159.0	1159.0	1159	1159	0.0	cuLinkDestroy
0.0	348	1	348.0	348.0	348	348	0.0	cuDeviceGetU...
0.0	226	1	226.0	226.0	226	226	0.0	cuModuleGetL...

```
...
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
** CUDA GPU Kernel Summary (gpukernsum):
```

Omitted: Avg, Med, Min, Max, StdDev

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** CUDA GPU Kernel Summary (gpukernsum):
```

Omitted: Avg, Med, Min, Max, StdDev

Time (%)	Total Time (ns)	Instances ...	GridXYZ	BlockXYZ	Name
-----	-----	-----	-----	-----	-----
100.0	27296	1 ...	977 1 1	1024 1 1	cupy::__main__::double_kernel...

```
...
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** CUDA GPU Kernel Summary (gpukernsum):
```

Omitted: Avg, Med, Min, Max, StdDev

Time (%)	Total Time (ns)	Instances	...	GridXYZ			BlockXYZ			Name
-----	-----	-----	...	-----	-----	-----	-----	-----	-----	-----
100.0	27296	1	...	977	1	1	1024	1	1	cupy::__main__::double_kernel...
...										

Blocks per grid

Threads per block

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** GPU MemOps Summary (by Time) (gpumemtimesum):
```

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
56.7	4361985	2	2180992.5	2180992.5	1315127	3046858	1224518.7	[CUDA memcpy DtoH]
43.3	3336073	2	1668036.5	1668036.5	1666069	1670004	2782.5	[CUDA memcpy HtoD]

D = Device = GPU

H = Host = CPU

```
** GPU MemOps Summary (by Size) (gpumemsizesum):
```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
16.000	2	8.000	8.000	8.000	8.000	0.000	[CUDA memcpy DtoH]
16.000	2	8.000	8.000	8.000	8.000	0.000	[CUDA memcpy HtoD]

```
...
```

CUDA Profiling with nsys

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** GPU MemOps Summary (by Time) (gpumemtimesum):
```

D = Device = GPU

H = Host = CPU

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
56.7	4361985	2	2180992.5	2180992.5	1315127	3046858	1224518.7	[CUDA memcpy DtoH]
43.3	3336073	2	1668036.5	1668036.5	1666069	1670004	2782.5	[CUDA memcpy HtoD]

```
** GPU MemOps Summary (by Size) (gpumemsizesum):
```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
16.000	2	8.000	8.000	8.000	8.000	0.000	[CUDA memcpy DtoH]
16.000	2	8.000	8.000	8.000	8.000	0.000	[CUDA memcpy HtoD]

```
...
```

CUDA Profiling with nsys

```
import numpy as np
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]

...
d_x = cuda.to_device(x)
d_y = cuda.device_array_like(d_x)

# Define compute grid and run
tpb = 1024
bpg = (n + (tpb - 1)) // tpb
double_kernel[bpg, tpb](x, y)

y = d_y.copy_to_host()
```

double.py

```
$ nsys profile -o manual_prof python double.py
WARNING: CPU IP/backtrace sampling not supported, disabling.
Try the 'nsys status --environment' command to learn more.
```

...

Generating '/tmp/nsys-report-c68f.qdstrm'

[1/1] [=====100%] manual_prof.nsys-rep

Generated:

/zhome/9d/d/98006/Documents/CUDA/manual_prof.nsys-rep

Manual memory transfer

CUDA Profiling with nsys

```
$ nsys stats manual_prof.nsys-rep
```

```
...
```

```
** GPU MemOps Summary (by Time) (gpumemtimesum):
```

D = Device = GPU

H = Host = CPU

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
-----	-----	-----	-----	-----	-----	-----	-----	-----
66.0	3093386	1	3093386.0	3093386.0	3093386	3093386	0.0	[CUDA memcpy DtoH]
34.0	1597044	1	1597044.0	1597044.0	1597044	1597044	0.0	[CUDA memcpy HtoD]

```
** GPU MemOps Summary (by Size) (gpumemsizesum):
```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
-----	-----	-----	-----	-----	-----	-----	-----
8.000	1	8.000	8.000	8.000	8.000	0.000	[CUDA memcpy DtoH]
8.000	1	8.000	8.000	8.000	8.000	0.000	[CUDA memcpy HtoD]

```
...
```

CUDA Profiling with nsys

```
$ nsys stats manual_prof.nsys-rep
```

```
...
```

```
** GPU MemOps Summary (by Time) (g...
```

Time (%)	Total Time (ns)	Count
66.0	3093386	1
34.0	1597044	1

```
** GPU MemOps Summary (by Size) (g...
```

Total (MB)	Count	Avg (MB)	Med (MB)
8.000	1	8.000	8.000
8.000	1	8.000	8.000

```
...
```

```
$ nsys stats double_prof.nsys-rep
```

```
...
```

```
** GPU MemOps Summary (by Time)
```

Time (%)	Total Time (ns)	Count
56.7	4361985	2
43.3	3336073	2

```
** GPU MemOps Summary (by Size)
```

Total (MB)	Count	Avg (MB)	Med (MB)
16.000	2	8.000	8.000
16.000	2	8.000	8.000

```
...
```

D = Device = GPU

H = Host = CPU

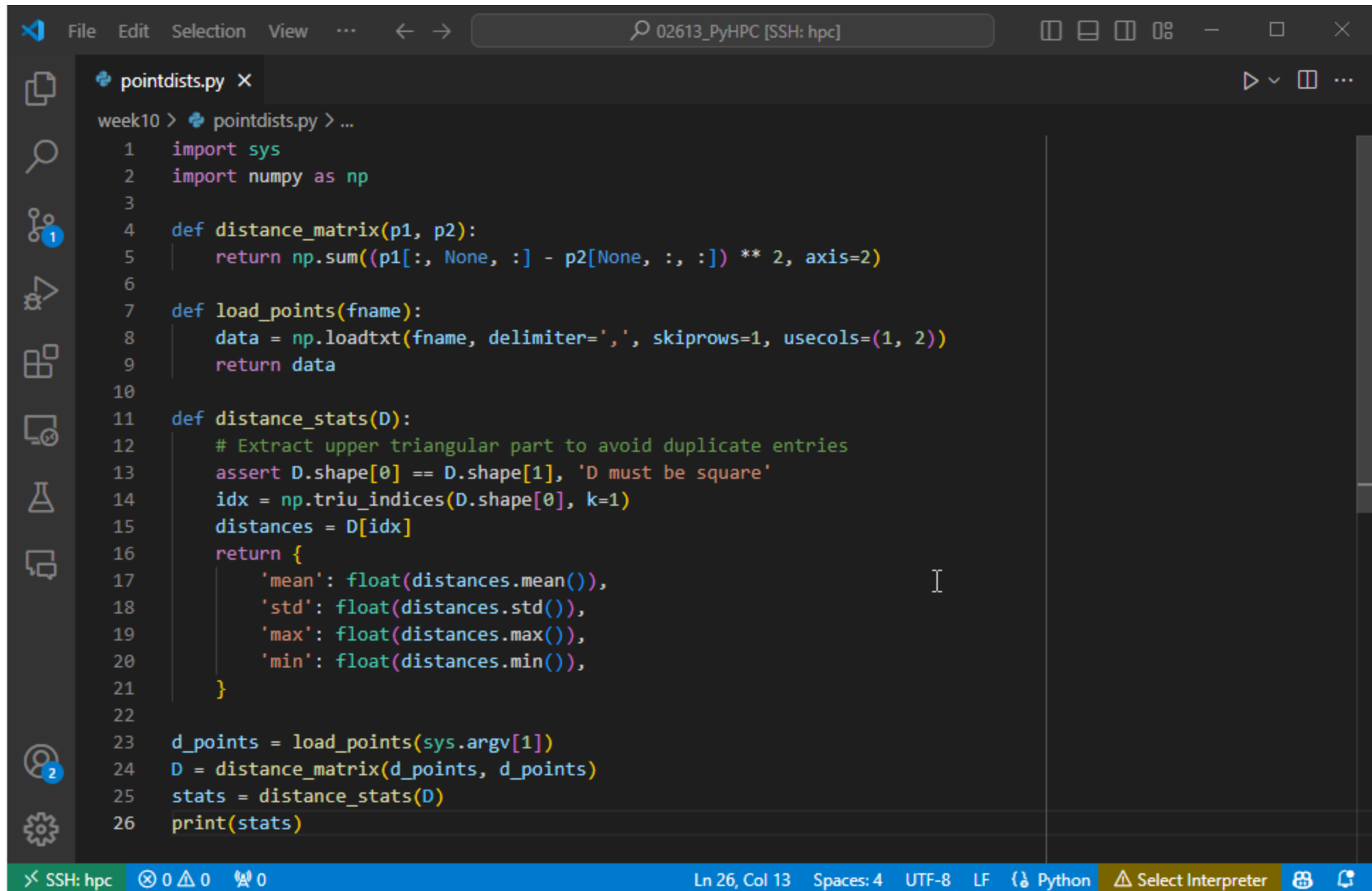
StdDev (ns)	Operation
0.0	[CUDA memcpy DtoH]
0.0	[CUDA memcpy HtoD]

Operation
CUDA memcpy DtoH]
CUDA memcpy HtoD]



CuPy

NumPy/SciPy-compatible Array Library for GPU-accelerated Computing with Python



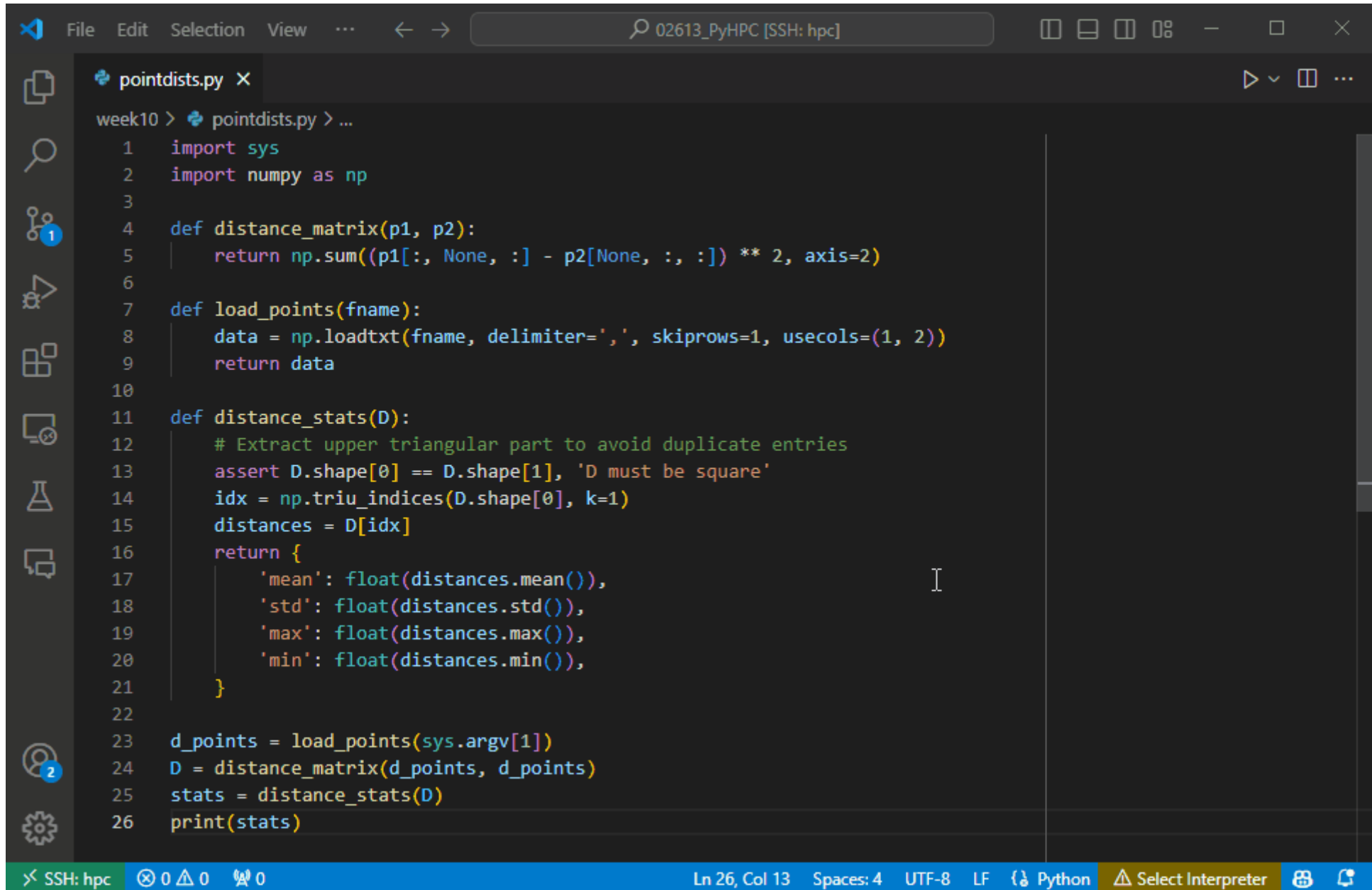
```
File Edit Selection View ... 02613_PyHPC [SSH: hpc]
```

pointdists.py X

week10 > pointdists.py > ...

```
1 import sys
2 import numpy as np
3
4 def distance_matrix(p1, p2):
5     return np.sum((p1[:, None, :] - p2[None, :, :]) ** 2, axis=2)
6
7 def load_points(fname):
8     data = np.loadtxt(fname, delimiter=',', skiprows=1, usecols=(1, 2))
9     return data
10
11 def distance_stats(D):
12     # Extract upper triangular part to avoid duplicate entries
13     assert D.shape[0] == D.shape[1], 'D must be square'
14     idx = np.triu_indices(D.shape[0], k=1)
15     distances = D[idx]
16     return {
17         'mean': float(distances.mean()),
18         'std': float(distances.std()),
19         'max': float(distances.max()),
20         'min': float(distances.min()),
21     }
22
23 d_points = load_points(sys.argv[1])
24 D = distance_matrix(d_points, d_points)
25 stats = distance_stats(D)
26 print(stats)
```

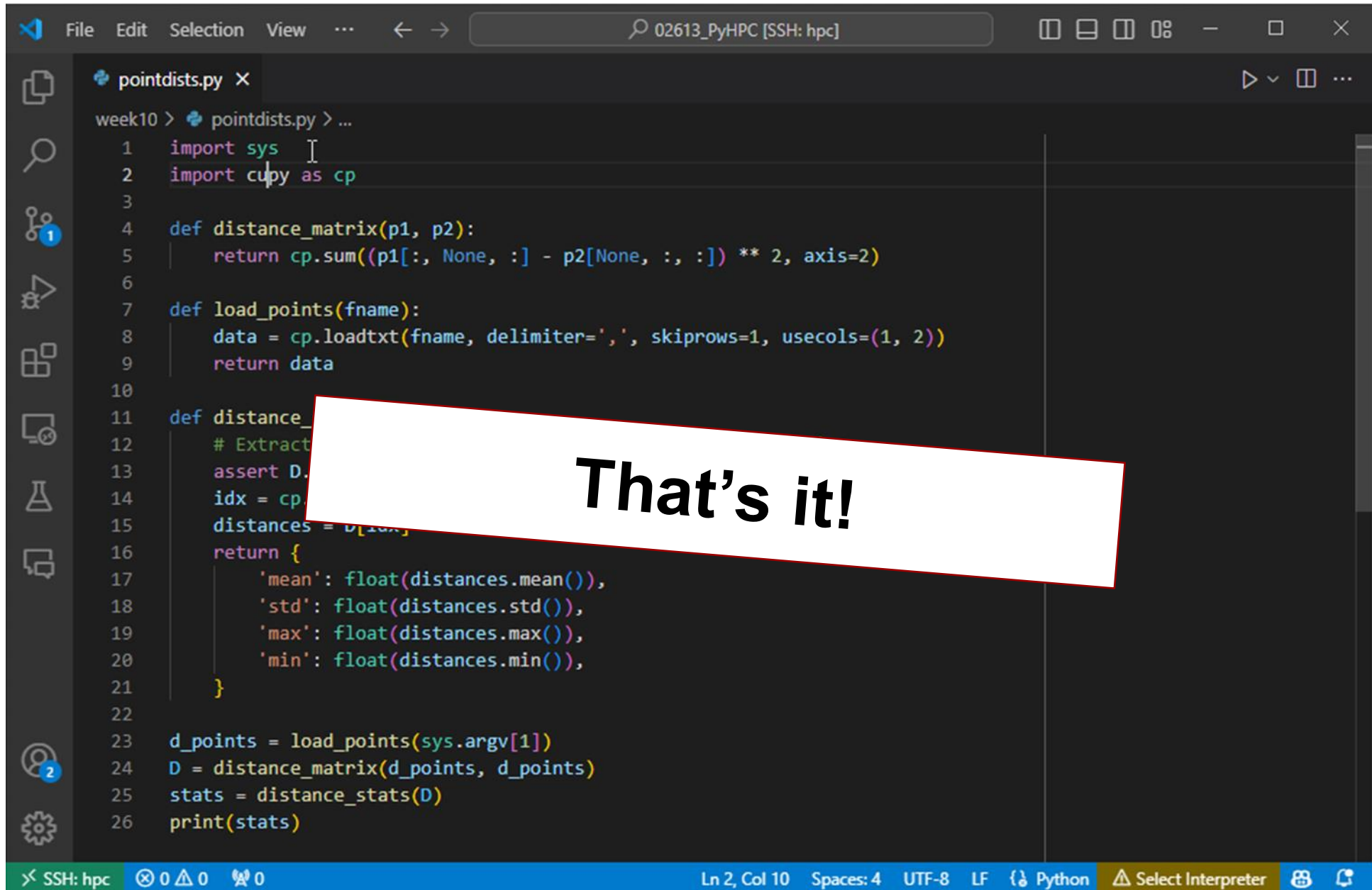
SSH: hpc 0 0 0 Ln 26, Col 13 Spaces: 4 UTF-8 LF Python Select Interpreter



```
File Edit Selection View ... 02613_PyHPC [SSH: hpc]

pointdists.py X
week10 > pointdists.py > ...
1 import sys
2 import numpy as np
3
4 def distance_matrix(p1, p2):
5     return np.sum((p1[:, None, :] - p2[None, :, :]) ** 2, axis=2)
6
7 def load_points(fname):
8     data = np.loadtxt(fname, delimiter=',', skiprows=1, usecols=(1, 2))
9     return data
10
11 def distance_stats(D):
12     # Extract upper triangular part to avoid duplicate entries
13     assert D.shape[0] == D.shape[1], 'D must be square'
14     idx = np.triu_indices(D.shape[0], k=1)
15     distances = D[idx]
16     return {
17         'mean': float(distances.mean()),
18         'std': float(distances.std()),
19         'max': float(distances.max()),
20         'min': float(distances.min()),
21     }
22
23 d_points = load_points(sys.argv[1])
24 D = distance_matrix(d_points, d_points)
25 stats = distance_stats(D)
26 print(stats)
```

SSH: hpc 0 0 0 Ln 26, Col 13 Spaces: 4 UTF-8 LF Python Select Interpreter



```
pointdists.py X
week10 > pointdists.py > ...
1 import sys
2 import cupy as cp
3
4 def distance_matrix(p1, p2):
5     return cp.sum((p1[:, None, :] - p2[None, :, :]) ** 2, axis=2)
6
7 def load_points(fname):
8     data = cp.loadtxt(fname, delimiter=',', skiprows=1, usecols=(1, 2))
9     return data
10
11 def distance_stats(D):
12     # Extract statistics from the distance matrix
13     assert D.ndim == 2
14     idx = cp.arange(D.shape[0])
15     distances = D[idx, idx]
16     return {
17         'mean': float(distances.mean()),
18         'std': float(distances.std()),
19         'max': float(distances.max()),
20         'min': float(distances.min()),
21     }
22
23 d_points = load_points(sys.argv[1])
24 D = distance_matrix(d_points, d_points)
25 stats = distance_stats(D)
26 print(stats)
```

SSH: hpc 0 0 0 Ln 2, Col 10 Spaces: 4 UTF-8 LF Python Select Interpreter

CuPy

- Built as a drop-in replacement for NumPy and SciPy
- Runs on operations on GPU – fast!

```
$ # NumPy
$ time python points.py largedata.csv
...

real    0m6.802s
user    0m5.373s
sys     0m1.403s
$
$ # CuPy
$ time python points.py largedata.csv
...

real    0m1.340s
user    0m0.956s
sys     0m0.234s
```



CuPy

- Built as a drop-in replacement for NumPy and SciPy
- Runs on operations on GPU – fast!
- Can also move from CPU to GPU from NumPy
- Can also be GPU input for Numba kernels
- ... and more!

```
>>> import numpy as np
>>> import cupy as cp
>>>
>>> x_cpu = np.zeros(100)
>>> x_gpu = cp.asarray(x_cpu) # H to D
>>> y_cpu = x_gpu.get()       # D to H
>>> z_cpu = cp.asnumpy(x_gpu) # D to H
...
>>> double_kernel[bpg, tpb](x_gpu, y_gpu)
```

Today's exercise

GPU profiling with Numba and CuPy

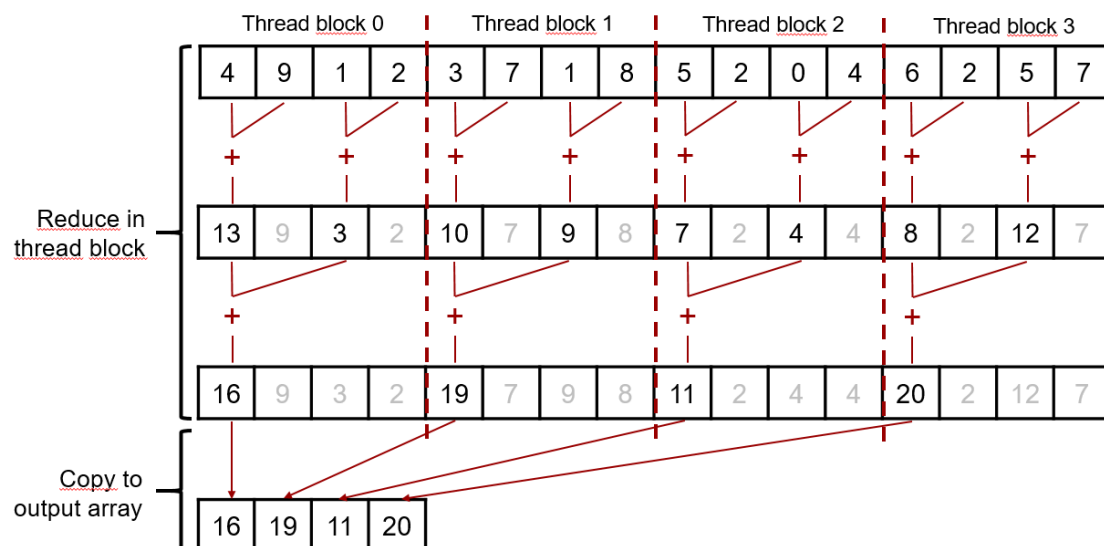
- GPU reduction and optimization

Profile both with nsys

- Convert previous Haversine distance code to CuPy

Useful concepts

Block reduce



Shared memory

```
cuda.shared.array(N, dtype=data.dtype)
```

CuPy

```
import cupy as cp
```

Change to GPU work node

```
voltash
sxm2sh
a100sh
```

GPU queue

```
c02613
```

Must add bsub option:

```
#BSUB -gpu "num=1:mode=exclusive_process"
```

Submit job script

```
bsub < submit.sh
```

Max wall time: 30 minutes

Min nr. of cores: 4